

Team GitHub Guide

How our 6-person team works together on the HR System project

Step by step, in simple words, for beginners

Our team repository

Open in the browser: <https://github.com/Group2Orange/Project-1>

Address used in commands: <https://github.com/Group2Orange/Project-1.git>

1. The big picture

You only need to know 7 words. Read them once, they will make sense when you use them.

Word	What it means
Git	A program on your computer. It remembers every version of your files, like an "undo history" for the whole project.
GitHub	A website that keeps our project online, so all 6 of us share the same files.
Repository	The project folder that Git watches. Ours is called Project-1 .
Branch	Your own copy of the project to work on. You can change things without breaking anyone else's work.
Commit	A saved snapshot of your work, with a short message that says what you did.
Push / Pull	Push = upload your commits to GitHub. Pull = download your teammates' work.
Pull Request	A request that says: "Please add my work to the main project." The team leader checks it, then adds it.



git clone = the very first copy of the project (only once)

2. The 7 team rules

These rules are what stop us from overwriting each other's work. Please follow them all.

- 1 Never work on main.**
main is the finished, shared version. You always work on your own branch.
- 2 One screen = one branch.**
Name it **screen/** + group folder + screen folder, joined with dashes. Example: `employee/new-request` becomes `screen/employee-new-request`.
- 3 Only edit your own screen's folder.**
If you own the Submit Request screen, you only change files inside `employee/new-request/`.
- 4 Shared files belong to the leader.**
Do not edit `index.html`, `README.md` or `guides/`. Ask Wadea, he will do it or review your change.
- 5 Start every work session with a pull.**
Run `git switch main` then `git pull` before you begin.
- 6 Commit often, push before you stop.**
Pushing is your backup. Work that is on GitHub cannot be lost.
- 7 Ask before doing anything scary.**
Never run `git push --force` or `git reset --hard` unless the leader tells you to.

TIP: Don't panic if you make a mistake. Git keeps history, so almost everything can be fixed. Stop, don't delete any folders, and ask the team.

3. Writing good commit messages

A commit message is a short note to your team. Say what you did, in a few words, starting with a verb.

Good	Bad
Add employee table to directory screen	stuff
Style leave request form buttons	update
Fix nav bar link on payslip screen	asdf

4. One-time setup (do this once, on your own computer)

IMPORTANT: Do step 5 **only after Wadea says "the project is uploaded"**. Before that, the repository only has empty starter files.

1

Make a free GitHub account

Go to **github.com** and click **Sign up**. Then send your GitHub **username** to Wadea so he can invite you to the team.

2

Accept the invitation

You will get an email from GitHub. Click the button in it, or open github.com/orgs/Group2Orange/invitation and click **Join**.

3

Install Git

Download it from **git-scm.com/downloads**, run the installer and just click **Next** until it finishes. Then open a **new** terminal and check that it works:

```
git --version
```

You should see something like **git version 2.x**.

4

Tell Git who you are (only once)

Use the same email as your GitHub account.

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

5

Copy the project to your computer (clone)

Pick a folder where you keep projects (for example Documents). In File Explorer, right-click an empty space and choose **Open in Terminal**. Then run:

```
git clone https://github.com/Group2Orange/Project-1.git
cd Project-1
code .
```

git clone makes a new folder called **Project-1** with the whole project. **cd Project-1** goes inside it. **code .** opens it in VS Code. **Always work inside this Project-1 folder.**

The first time you push, a window opens and asks you to sign in to GitHub. Sign in there. That is normal.

6

Check that it worked

```
git status
```

It should say **On branch main** and **nothing to commit, working tree clean**. You are ready!

5. Team leader only: first upload and GitHub settings

Part A: upload the project for the first time. The repository already has three empty starter files (**CSS/style.css**, **HTML/index.html**, **JS/script.js**) and a one-line **README**. We replace them with our project.

1 Clone the repository next to your current folder

```
cd E:\
git clone https://github.com/Group2Orange/Project-1.git
```

2 Copy the project into it

Open **E:\HRSysOrange**, press **Ctrl+A** to select everything, copy, and paste it into **E:\Project-1**. When Windows asks about **README.md**, choose **Replace the file in the destination**. Then delete the old empty folders **CSS**, **HTML** and **JS** inside **Project-1**.

3 Check, save and upload

```
cd Project-1
git status
git add .
git commit -m "Add project structure"
git push
```

From now on, Wadea also works inside **Project-1**, like everyone else. Keep **HRSysOrange** as a backup for now.

Part B: let the team in.

4 Invite everyone and give them Write access

Go to **github.com/orgs/Group2Orange > People > Invite member** and add each teammate's username. Then open the repository > **Settings > Collaborators and teams** and give them the **Write** role. By default new members can only read. Without Write they cannot push.

Part C: protect main (do this after the first upload).

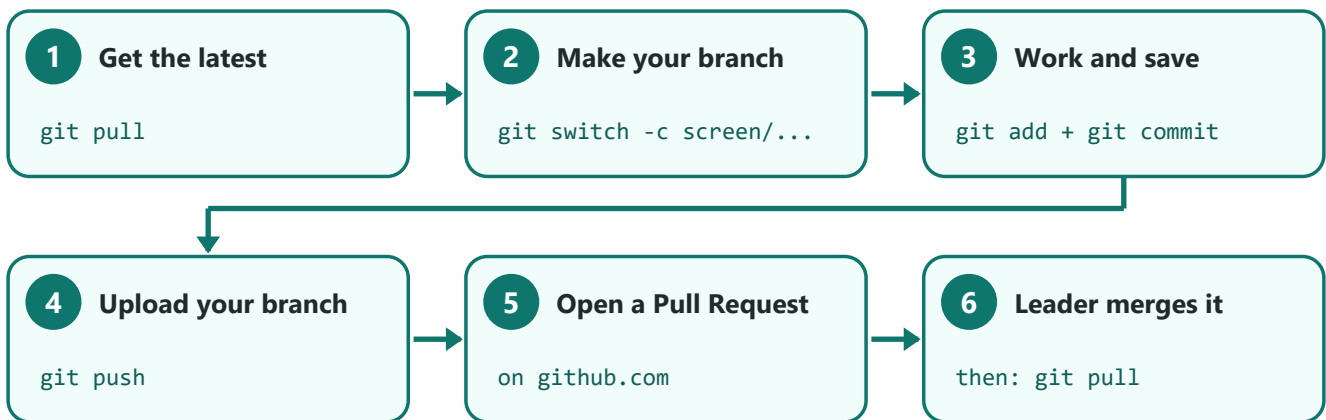
5 Only allow changes through Pull Requests

Repository > **Settings > Rules > Rulesets > New ruleset > New branch ruleset**. Name it "Protect main", set Enforcement to **Active**, add the target **Include default branch**, and tick **Require a pull request before merging** (set required approvals to 0 so you can merge yourself). Click **Create**. Button names may look a little different if GitHub changes its design.

TIP: Reviewing a Pull Request: open the **Files changed** tab and check that the person only changed files inside their own screen folder. If yes, click **Merge pull request**, then **Confirm merge**, then **Delete branch**.

6. Every day: the workflow for one screen

Follow these steps every time. The example builds the **Submit Request** screen, in the folder `employee/new-request/`. Use your own screen name instead.



1 Get the latest version of main

Do this every time before you start. It downloads what your teammates finished.

```
git switch main
git pull
```

2 Create your own branch

Name it **screen/** + the group folder (common, hr or employee) + your screen folder, joined with dashes (for example **common/home** becomes **screen/common-home**). The **-c** means "create".

```
git switch -c screen/employee-new-request
```

Check which branch you are on any time with `git branch`. The one with a star (*) is your current branch.

3 Build your screen

Open your screen's folder in VS Code and edit your **.html**, **.css** and **.js** files. Save with **Ctrl+S**. Preview with **Live Server** (right-click the **.html** file > Open with Live Server). Only touch files in your own folder.

4 Check what you changed

```
git status
```

You should only see files from your screen's folder. **If you see other files, stop and ask Wadea.**

5 Save a snapshot (commit)

git add chooses the files. **git commit** saves them with a message. Write what you did in a few words.

```
git add employee/new-request
git commit -m "Add request form and buttons"
```

WARNING: Never use `git add .` — it adds everything, even files you did not mean to add. Always name your own folder.

6

Upload your branch to GitHub (push)

The first time you push a new branch, use this:

```
git push -u origin screen/employee-new-request
```

After that, on the same branch, just type `git push`. Push at the end of every work session, it is your backup.

7

Open a Pull Request on GitHub

- Open <https://github.com/Group2Orange/Project-1>. You will see a yellow banner with your branch name. Click **Compare & pull request**.
- Write a short title, for example **Add Leave Requests screen**. Add one line about what you built (a screenshot helps).
- Check that it says **base: main** and **compare: your branch**, then click **Create pull request**.
- Tell Wadea in the team chat. If he asks for changes, edit your files, commit and push again on the same branch. The Pull Request updates by itself.

8

After Wadea merges it: clean up and go again

Your work is now part of main. Get it and delete your finished branch, then start your next screen from step 2.

```
git switch main
git pull
git branch -d screen/employee-new-request
```

7. Getting your teammates' new work into your branch

Usually you do **not** need this, because everyone edits different folders. Use it only when something shared changed (for example the leader added a file you need) and you want it in your branch:

```
git switch main
git pull
git switch screen/employee-new-request
git merge main
```

8. What is a conflict, and how to fix it

A conflict happens when two people change the **same lines of the same file**. Git cannot choose, so it asks you. Because each of us owns different folders, conflicts should be rare. If one happens, do not worry:

1

Git tells you which file has a conflict

After `git merge main` you will see a message like **CONFLICT (content): Merge conflict in ...**. Open that file in VS Code.

2

Choose what to keep

VS Code shows the two versions between marker lines and gives you buttons above them: **Accept Current Change** (yours), **Accept Incoming Change** (theirs) or **Accept Both Changes**. Pick one. Make sure the marker lines (<<<<<<, =====, >>>>>>) are gone, then save.

3

Finish the merge

Replace the file name with the file you fixed:

```
git add path/to/the/file
git commit -m "Fix merge conflict"
git push
```

TIP: Not sure which version to keep? Ask the person who owns the other change. Do not guess on shared files.

9. Problems and quick fixes

Problem	Fix
<code>fatal: not a git repository</code>	You are in the wrong folder. Go into the project: <code>cd Project-1</code> .
<code>Please tell me who you are</code>	Do setup step 4 (the two git config commands).
I changed files while I was on main	Before you commit, run <code>git switch -c screen/your-screen</code> . Your changes come with you to the new branch. Then commit as usual.
<code>rejected ... fetch first</code>	Someone pushed before you. Run <code>git pull</code> and then <code>git push</code> again.
<code>403</code> or <code>Permission denied</code> when pushing	You do not have Write access yet. Check that you accepted the invitation, then ask Wadea to give you the Write role.
Which branch am I on?	Run <code>git branch</code> . The one with a star (*) is your branch.
I want to undo my changes in one file	<code>git restore path/to/file</code> Warning: this deletes your unsaved changes in that file forever.
I am lost or scared I broke something	Stop. Do not delete any folders. Run <code>git status</code> , take a screenshot and send it to the team chat. Anything you already pushed is safe on GitHub.

10. Cheat sheet

Command	What it does
<code>git clone <address></code>	Copy the project to your computer (only once). The address is at the top of page 1.
<code>git status</code>	Show what you changed. Use it all the time.
<code>git branch</code>	Show your branches. The star (*) is where you are.
<code>git switch main</code>	Go to the main branch.
<code>git pull</code>	Download the latest work from GitHub.
<code>git switch -c screen/name</code>	Create a new branch and go to it.
<code>git add folder/</code>	Choose the files to save (name your own folder).
<code>git commit -m "message"</code>	Save a snapshot with a message.
<code>git push</code>	Upload your commits to GitHub.
<code>git push -u origin screen/name</code>	Upload a new branch for the first time.
<code>git merge main</code>	Bring the latest main into your branch.
<code>git branch -d screen/name</code>	Delete a finished branch on your computer.

TIP: Want to know what every folder and file is for, and how to build a screen? Read the **Teammate Handbook** (<guides/Teammate-Handbook.pdf>).